

Iterative Improvement and Variable Neighborhood Descent for the Linear Ordering Problem

Rahman Tektas

542557

ULB

rahman.tektas@ulb.be

ABSTRACT

This report studies local search algorithms for the Linear Ordering Problem (LOP). First, iterative improvement algorithms were implemented using two pivoting rules, first-improvement and best-improvement, combined with the three neighborhoods transpose, exchange, and insert. Two initialization methods were considered: a random permutation and the Chenery–Watanabe (CW) heuristic. This yields twelve algorithmic combinations. Second, two Variable Neighborhood Descent (VND) variants were implemented, using the neighborhood orders transpose–exchange–insert and transpose–insert–exchange, with CW initialization and first-improvement only. The algorithms were evaluated on the benchmark instances provided for the course using the average percentage deviation from the best known solutions and the total computation time. Statistical comparisons were carried out with paired Student t-tests and Wilcoxon signed-rank tests in R.

1 INTRODUCTION

The Linear Ordering Problem (LOP) is a combinatorial optimization problem defined on a square matrix C . The goal is to find a permutation of the rows and columns that maximizes the sum of matrix entries above the diagonal in the reordered matrix. Because the problem is hard, heuristic methods are often used to obtain good solutions in reasonable time.

In this work, two families of local search methods are studied. In Exercise 1.1, iterative improvement algorithms are compared under different pivoting rules, neighborhoods, and initialization methods. In Exercise 1.2, Variable Neighborhood Descent (VND) is used to combine several neighborhoods within one algorithm. The objective is to study how these design choices affect solution quality and runtime.

2 PROBLEM DESCRIPTION

Let C be an $n \times n$ matrix, where c_{ij} denotes the entry in row i and column j . A solution of the LOP is a permutation π of the indices $\{1, \dots, n\}$. The objective function is

$$f(\pi) = \sum_{i=1}^n \sum_{j=i+1}^n c_{\pi(i)\pi(j)}.$$

The goal is to find a permutation π that maximizes $f(\pi)$. Three neighborhoods were considered in this work:

- **Transpose:** swap two adjacent elements.
- **Exchange:** swap two elements at arbitrary positions.
- **Insert:** remove one element from a position and insert it at another position.

Transpose is the smallest neighborhood, while exchange and insert allow larger changes to the ordering.

3 MATERIALS AND METHODS

3.1 Iterative Improvement Algorithms

For exercise 1.1, iterative improvement algorithms were implemented using :

- two pivoting rules:
 - first-improvement
 - best-improvement
- three neighborhoods:
 - transpose
 - exchange
 - insert
- two initialization methods:
 - random permutation
 - Chenery–Watanabe (CW) heuristic

This yields $2 \times 3 \times 2 = 12$ combinations.

3.2 Incremental update

To avoid unnecessary computations, incremental updates of the evaluation function value were implemented for each neighborhood. This makes it possible to evaluate the objective function right after a move, avoiding the recomputation of the full objective function from scratch.

Let

$$\pi = (\pi_1, \pi_2, \dots, \pi_n)$$

be the current permutation, and let

$$f(\pi) = \sum_{h=1}^n \sum_{k=h+1}^n c_{\pi_h \pi_k}.$$

Denote by $\Delta = f(\pi') - f(\pi)$ the change in objective value after applying a move.

- **Transpose:** swapping adjacent elements at indices i and $i+1$:

$$\Delta = c_{\pi_{i+1}\pi_i} - c_{\pi_i\pi_{i+1}}.$$

- **Exchange:** swapping elements at indices i and j , with $i < j$:

$$\Delta = c_{\pi_j\pi_i} - c_{\pi_i\pi_j} + \sum_{k=i+1}^{j-1} (c_{\pi_k\pi_i} - c_{\pi_i\pi_k} + c_{\pi_j\pi_k} - c_{\pi_k\pi_j}).$$

- **Insert:** removing the element at index i and inserting it at index j :
if $i < j$,

$$\Delta = \sum_{k=i+1}^j (c_{\pi_k\pi_i} - c_{\pi_i\pi_k}),$$

and if $i > j$,

$$\Delta = \sum_{k=j}^{i-1} (c_{\pi_i \pi_k} - c_{\pi_k \pi_i}).$$

3.3 Chenery–Watanabe Heuristic

This heuristic constructs the solution each row at a time, at each step selecting the most attractive remaining row, which is the one that maximizes the sum of elements having an influence on objective function.

The attractiveness of a row i at construction step s can be defined as such :

$$\sum_{j=s+1}^n c_{\pi(i)j}.$$

This means the heuristic prefers the rows that have the highest remaining contribution to the objective function, so this method is intended to provide better initial solutions than an uniformed random starting point.

3.4 Variable Neighborhood Descent

For exercise 1.2, 2 Variable Neighborhood Descent (VND) variants were implemented, using the neighborhood orders:

- (1) transpose $\rightarrow$ exchange $\rightarrow$ insert
- (2) transpose $\rightarrow$ insert $\rightarrow$ exchange

In both cases, only the first-improvement pivoting rule was used, and the CW heuristic was used for initialization.

The VND algorithm starts from the first neighborhood, and if an improving move is found, it applies it and restarts from the first neighborhood. If no improving move exists it just moves to the next neighborhood, and so on. The algorithm terminates when no improving move exists in any neighborhood.

4 RESULTS

The benchmark instances used in the experiments were the provided LOP instances of sizes 150 and 250. Each algorithm was applied once to each instance, and a random seed was set for reproducibility and fair comparison. For each run, two values were recorded:

- the final solution cost
- the computation time

Below is a summary table, where solution quality is measured using the relative percentage deviation from the best known solution, defined as

$$\Delta_{ki} = 100 \cdot \frac{\text{best-known}_i - \text{cost}_{ki}}{\text{best-known}_i}.$$

With this convention, smaller values correspond to better solutions.

4.1 Exercise 1.1 : Summary of the 12 Iterative Improvement Algorithms

Table 1: Summary results for the 12 iterative improvement algorithms.

Algorithm Variant	Avg. Deviation (%)	Std. Dev.	Total Time (s)
cw best exchange	3.8465	0.6054	32.9845
cw best insert	2.2608	0.4065	33.7210
cw best transpose	17.0783	1.6065	0.2194
cw first exchange	2.9660	0.4613	186.7398
cw first insert	2.0164	0.4109	128.2090
cw first transpose	17.0863	1.6042	0.2148
random best exchange	3.6027	0.5048	35.8138
random best insert	2.3037	0.4635	37.9277
random best transpose	34.4701	3.8033	0.0035
random first exchange	2.8159	0.4810	228.0147
random first insert	2.0211	0.4507	189.8870
random first transpose	34.6058	3.7984	0.0024

4.2 Exercise 1.2 : Summary of the VND Algorithms

Table 2: Summary results for the two VND algorithms.

Method	Avg. Deviation (%)	Std. Dev.	Total Time (s)
vnd-tei	-2.0633	0.4191	109.6544
vnd-tie	-1.9959	0.4220	80.5438

4.3 Tests

A series of tests were performed to compare the algorithms, to answer the questions of Is there a statistically significant difference between the solution quality generated by the different algorithms?

The wilcoxon

The two VND variants were compared using paired Student t-tests and Wilcoxon signed-rank tests.

- Average deviation vnd-tei: -2.0633
- Average deviation vnd-tie: -1.9959
- Std deviation vnd-tei: 0.4191
- Std deviation vnd-tie: 0.4220
- Paired t-test p-value: 0.2136
- Wilcoxon p-value: 0.4169

Although the transpose–insert–exchange variant obtained a slightly better average deviation than the transpose–exchange–insert variant, both statistical tests returned p-values greater than 0.05. Therefore, no statistically significant difference in solution quality was observed between the two VND algorithms.

5 DISCUSSION

Report on the how you interpret the results and how your interpretation may differ.

Explain what you think should be done as a future analysis, implementation or algorithm to advance the results discussed in the original paper.

Page limit: 1

6 CONCLUSIONS

Some paragraphs for summarizing the main findings of the report and your experience in reproducing the work.

REFERENCES